

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and
Commerce
Department of Information Technology (Bsc.IT Semester III)
SCALA for DS
Practical

<i>Roll No. : SY50</i>	<i>Name: Nitesh Yadav</i>
<i>Class:SYIT</i>	<i>Date of Submission :01/09/2026</i>

25). Handle missing values in a dataset. Replace missing values with the column mean.

Input:

```
object Scala25 extends App {  
  
  val sales: List[java.lang.Double] =  
    List(100.0, 200.0, null, 300.0, null)  
  
  // Remove null values  
  val v = sales.filter(_ != null)  
  
  // Calculate mean  
  val mean = v.map(_.toDouble).sum / v.length  
  
  println("Mean Sales: " + mean)  
  
  // Replace null values with mean  
  val n = sales.map { value =>  
    if (value == null) mean  
    else value  
  }  
  
  println("Original Data:")  
  println(sales.mkString(", "))  
  
  println("After Replacing Missing Values:")  
  println(n.mkString(", "))  
}
```

Output:

```
Mean Sales: 200.0
Original Data:
100.0, 200.0, null, 300.0, null
After Replacing Missing Values:
100.0, 200.0, 200.0, 300.0, 200.0

Process finished with exit code 0
|
```

26).Perform Logistic Regression using Breeze

Perform logistic regression using Breeze. Classify a dataset with binary labels.

Input:

```
import breeze.linalg._
import breeze.numbers._

object Scala25 extends App {
  // Dataset
  val x = DenseVector(1.0, 2.0, 3.0, 4.0, 5.0)

  // Binary labels
  val y = DenseVector(0.0, 0.0, 0.0, 1.0, 1.0)

  // Initial weight and bias
  var weight = 0.0
  var bias = 0.0

  // Sigmoid function
  def sigmoid(z: Double): Double = {
    1.0 / (1.0 + math.exp(-z))
  }

  // Train the model
  for (i <- 1 to 1000) {
    for (j <- 0 until x.length) {
```

```

    val prediction = sigmoid(weight * x(j) + bias)
    val error = prediction - y(j)

    weight = weight - 0.01 * error * x(j)
    bias = bias - 0.01 * error
  }
}

// Classify the data
println("Predictions:")

for (i <- 0 until x.length) {
  val probability = sigmoid(weight * x(i) + bias)
  val result = if (probability >= 0.5) 1 else 0

  println(
    "Input = " + x(i) +
    ", Probability = " + probability +
    ", Class = " + result
  )
}

```

Output:

```

Predictions:
Input = 1.0, Probability = 0.0515501057576029, Class = 0
Input = 2.0, Probability = 0.16412826954644827, Class = 0
Input = 3.0, Probability = 0.4149888799701166, Class = 0
Input = 4.0, Probability = 0.7193154698480789, Class = 1
Input = 5.0, Probability = 0.9025176083649726, Class = 1

Process finished with exit code 0

```